

ENGINEERING ONBOARDING · CONFIDENTIAL

NxtSft

Enterprise Real-Estate Platform — PRD, Architecture & Build State

Document type: New-developer deep-dive (PRD + status + full flows)

Generated: 2026-07-06 · main @ `b3a762a`

Stack: Next.js 15 · tRPC v11 · Prisma 7 · PostgreSQL 16 · Tailwind 4

Phase: v1.0 Demo / Prototype — live DB, wired portals, security-hardened

Audience: Any engineer joining the project cold

Contents

1 · Product Overview (PRD)	9 · Core User Flows
2 · Business Verticals	10 · What's Done ✓
3 · Roles & Personas	11 · What's Remaining □
4 · Tech Stack & Architecture	12 · Known Issues & Tech Debt
5 · Monorepo Layout	13 · Local Setup & Dev Workflow
6 · Data Model (Prisma)	14 · Deployment
7 · API Surface (tRPC + REST)	15 · First-Week Onboarding Path
8 · Auth & Security Model	

How to read this doc. Sections 1–3 are the *what & why* (product). Sections 4–8 are the *how* (engineering). Section 9 ties them together with end-to-end flows. Sections 10–12 are the honest state of the build. Sections 13–15 get you running and productive.

39

PRISMA MODELS

25

TRPC ROUTERS

6

ROLE PORTALS

6

BUSINESS VERTICALS

~50

PUBLIC ROUTES

DONE

shipped & wired

PARTIAL

backend or UI only

TODO

not started

1 · Product Overview (PRD)

NxtSft is an enterprise real-estate marketplace for the Indian market. It connects **property seekers** (buyers/renters) with **owners, agents, and builders**, and runs a full internal **CRM + sales operation** on top – lead capture, assignment, site visits, commissions, subscriptions, and credit-gated contact unlocks.

1.1 Product thesis

- **Two-sided marketplace + operations tool in one.** The public site is the demand/supply funnel; the six role portals are the machine that works the funnel.
- **Contact-unlock economy.** Seeing a listing is free; unlocking the owner/agent phone number costs a **credit**. Credits come from **plans/subscriptions** (paid) – this is the primary monetization.
- **Multi-vertical directory.** Beyond resale/rental property, the platform lists Builders, Projects, Home Interior designers, Decor stores, and PG/Co-living – each a mini-marketplace with its own browse/detail/lead flow.
- **Lead lifecycle is first-class.** Every inquiry becomes a **Lead** that flows through a CRM kanban (New → Hot → Warm → Cold → Converted / Lost), gets assigned to sales reps, and drives site visits and commissions.

1.2 Monetization surfaces

Surface	How it earns	Status
Credit-gated contact unlock	Buyer spends a credit to reveal owner/agent contact	DONE
Subscription plans	Seeker / Owner-Rent / Owner-Sell / Reseller plan tiers grant credits + perks	DONE
Payment gateway	Switchable Razorpay ⇌ PayU (chosen via <code>SiteSetting</code> , default Razorpay)	PARTIAL – needs live keys
Listing boost / featured	Paid promotion of a listing (UI stub; toast only)	TODO
Referral rewards	₹ reward per referral type (<code>referrals.ts</code>)	PARTIAL

Current phase: v1.0 Demo / Prototype. The database is **live** (teammate-hosted VPS PostgreSQL), most portals read real data via `tRPC`, auth is verified end-to-end, and a full security-hardening pass (GOL-268) shipped. Remaining work is mostly *replacing residual fake-data panels with `tRPC`, wiring live payment keys, and media upload.*

2 · Business Verticals

Each vertical follows the same shape: a public **browse** hub, an SEO-rendered **detail** page (with JSON-LD), a **self-serve submit** path (lands as `pending` for admin review), favorites, view tracking, and credit-gated contact unlock.

Vertical	Routes	Core models	tRPC router
Property (resale/rent)	/properties, /properties/[slug], /list	Property, Location, Listing	properties, listings
Builders	/builders, /builders/[slug]	Builder, Project	builders, projects
Home Interiors	/interiors, /interiors/[slug], /interiors/list	InteriorDesigner, DesignerFavorite, InteriorDesignerView	interiorDesigners
Decor stores	/decor, /decor/[slug], /decor/list	DecorStore, DecorFavorite, DecorStoreView	decorStores
PG / Co-living	/pg	Property (type-tagged)	properties
Agents / Owners	/agents, /agents/[slug], /owners/[slug]	User (role=agent)	users.getAgents/getAgent

Vertical parity note. Property, Builders, Interiors and Decor are the "complete" verticals (browse + SEO detail + submit + admin CRUD). `/owners/[slug]` still renders from a static fixture (see §12). PG reuses Property with a type tag rather than its own model.

2.1 Supporting content pages

Marketing/legal/content routes: `/about`, `/contact`, `/pricing`, `/refer`, `/careers`, `/nri-guide`, `/nri-investment-guide`, `/terms`, `/privacy`, `/cookie-policy`, `/fraud-advisory`, `/refund`, `/unsubscribe`. Hero + some marketing content is CMS-driven via the `SiteSetting` KV pattern (key `home.hero`) with static fallback.

3 · Roles & Personas

Role is a string on `User.role`, default `user`. Enforced server-side by tRPC procedure guards and mirrored in the page-routing middleware.

Role	Portal	What they do
<code>user / home-seller</code>	<code>/user-portal</code>	Buyer or seller. Browse, save, unlock contacts, buy credits, manage own listings (sellers), KYC, site visits, support tickets.
<code>agent</code>	<code>(public profile)</code>	Listed in the agent directory; has a public <code>/agents/[slug]</code> profile.
<code>sales</code>	<code>/sales-portal</code>	Works assigned leads, click-to-call, schedules visits, tracks own commissions & reports (rep-scoped).
<code>supervisor</code>	<code>/supervisor-portal</code>	Sees all leads & reps, geographic visit calendar (Mapbox), team reports.
<code>support-admin</code>	<code>/support-portal</code>	Ticket queue, escalations, TAT reports, knowledge base.
<code>admin</code>	<code>/admin-portal</code>	Listings approval, CRM pipeline, subscriptions, team management, KYC review, credit-usage & buyer-activity analytics, campaigns, per-vertical CRUD.
<code>super-admin</code>	<code>/sa-portal</code>	Command dashboard, user/role management, security console, IP rules, permission matrix, plans manager, billing, AI model registry, CMS, audit trail, broadcast notifications.

3.1 tRPC procedure guards

Defined in `packages/trpc/src/server.ts`. A request carries an `httpOnly` session cookie; the guard re-verifies the token against the DB and checks role.

Guard	Requires
<code>publicProcedure</code>	No auth (anon-friendly; some read the optional session)
<code>protectedProcedure</code>	Any authenticated user
<code>staffProcedure</code>	Any staff role (sales/supervisor/support/admin/super-admin)
<code>supervisorProcedure</code>	supervisor or above
<code>adminProcedure</code>	admin or super-admin
<code>superAdminProcedure</code>	super-admin only

4 · Tech Stack & Architecture

Frontend

- **Next.js 15** App Router, React 19, Server Components
- **Tailwind CSS 4** + **Radix UI** primitives
- Branded UI kit (`ui/select` , skeletons, load-more)
- **Mapbox GL** (react-map-gl) for property + visit maps

Backend / API

- **trRPC v11** — end-to-end typed API, served at `/api/trpc`
- **REST v1** mirror at `/api/v1/*` (Bearer auth)
- **Fastify** app (`apps/api`) exists but is *not deployed*
- Route handlers run on Vercel Fluid Compute (Node)

Data

- **PostgreSQL 16** on teammate VPS (`187.77.185.220:5433`)
- **Prisma 7** — engine-free WASM query compiler via pg adapter
- Schema managed by `prisma db push` (**no migrations folder**)
- 39 models; JSONB + FTS; pgvector planned for semantic search

Planned / partial infra

- **Redis** cache + BullMQ jobs (architecture; not fully wired)
- **Cloudflare R2** media storage (media router ready; needs creds)
- **Typesense** search, PostHog analytics, Sentry (planned)
- **Resend/Twilio/WATI** email/SMS/WhatsApp (planned)

4.1 Deployment shape (important)

Only `apps/web` is deployed to Vercel. It hosts both the site *and* the API (via `/api/trpc` + `/api/v1/*`). The Fastify `apps/api` is not deployed. Vercel **Root Directory** = `apps/web` . Prisma client is generated on Vercel via the `packages/db` postinstall hook.

4.2 Why Prisma 7

Vercel repeatedly failed with `"Query Engine for rhel-openssl-3.0.x not found"` because file-tracing wouldn't bundle the native `.so.node` engine. Upgrading to **Prisma 7** (engine-free — a base64-embedded WASM query compiler runs through the pg adapter) removed the native binary entirely, making that class of error impossible.

5 • Monorepo Layout

Turborepo + pnpm workspaces. All commands run from the repo root.

Path	What lives there
apps/web	Next.js 15 frontend + the deployed API (route handlers). This is where 90% of work happens.
apps/api	Fastify server — not deployed ; its own type-check currently fails (pre-existing, see §12).
packages/db	Prisma schema (<code>prisma/schema.prisma</code>), seed (<code>prisma/seed.ts</code>), configured client (<code>client.ts</code>), <code>prisma.config.ts</code> .
packages/trpc	All 25 routers (<code>src/routers/*</code>), procedure guards + rate limiter (<code>server.ts</code>), payment helpers (<code>payu.ts</code>).
packages/shared	Zod validators, shared types, constants, <code>session-cookie.ts</code> (HMAC/hash helpers), <code>client-ip.ts</code> .
packages/config	Shared ESLint / Prettier / tsconfig.

5.1 Frontend internal structure (`apps/web/src`)

Folder	Contents
app/	~50 routes (public pages, 6 portals, <code>api/</code> handlers).
components/	home/, site/ (header/footer/WhatsApp), pricing/, portal/, ui/ (design system), per-portal tabs.
data/	Static fixtures (<code>static.ts</code> , <code>reports.ts</code> , <code>agents.ts</code> , <code>dummyNames.ts</code>) — being dismantled into tRPC. See <code>docs/static-data-inventory.md</code> .
lib/	<code>auth.tsx</code> (AuthProvider), <code>site.ts</code> , <code>map.ts</code> , <code>rera.ts</code> , <code>routes.ts</code> , <code>nonce.ts</code> , <code>propertyActivity.ts</code> .

Key reference doc: `docs/static-data-inventory.md` is the authoritative audit of every remaining hardcoded/fake data source, what consumes it, and the intended action (CMS / DB / ENV / DEDUP / DEAD / KEEP). Read it before touching any portal panel.

6 · Data Model (Prisma — 39 models)

Schema at `packages/db/prisma/schema.prisma` . Grouped by domain:

Domain	Models
Identity & auth	User · Session · AuditLog
Property core	Property · Location · Listing · PropertyView · PropertyEditRequest · Favorite · Review
Builders vertical	Builder · Project
Interiors vertical	InteriorDesigner · DesignerFavorite · InteriorDesignerView
Decor vertical	DecorStore · DecorFavorite · DecorStoreView
CRM & sales	Lead · Enquiry · SalesActivity · Commission · SiteVisit · Escalation · Ticket
Money	Plan · Subscription · Payment · CreditTransaction
Engagement	Notification · Message · SearchAlert · ReferralSubmission · UnsubscribeRequest
Compliance	KycDocument
Platform / CMS	SiteSetting · CmsPage · Campaign · ModelVersion

6.1 Central entity: Lead

The lead is the spine of the operations side. Status enum drives the CRM kanban: **New** → **Hot** → **Warm** → **Cold** → **Converted / Lost**. A lead is created from: a property inquiry form, a contact unlock, a scheduled visit request, or bulk import. It gets assigned to a sales rep, accrues `SalesActivity` , and on conversion drives a `Commission` .

6.2 Credit economy: CreditTransaction

Every unlock writes a `CreditTransaction(reason="contact_unlock", ...)` linking buyer → property/designer/decorStore. Admin analytics (`admin.creditUsage`) read this to show who spent which credit where. Credit balance is derived; plans top it up.

Schema workflow — read this. There is **no migrations folder**. The DB is shared. For additive changes use `prisma db push` . **Never run `migrate dev`** against the shared DB — it can wipe/reset. Password in the connection string contains `@` → must be percent-encoded as `%40` .

7 · API Surface

7.1 tRPC routers (25)

Mounted in `packages/trpc/src/routers/index.ts`, served from Next at `/api/trpc`.

Router	Key procedures
<code>auth</code>	<code>register</code> · <code>login</code> · <code>loginStaff</code> · <code>logout</code> · <code>me</code>
<code>properties</code>	<code>list</code> (infinite/featured) · <code>get</code> · <code>create</code> · <code>update</code> · <code>approve</code> · <code>toggleFeatured</code> · <code>unlockContact</code>
<code>listings</code>	self-serve listing management
<code>leads</code>	<code>create</code> · <code>list</code> · <code>get</code> · <code>updateStatus</code> · <code>addNote</code> · <code>assign</code> · <code>scheduleVisit</code> · <code>bulkAssign</code> · <code>stats</code>
<code>users</code>	<code>me</code> · <code>updateProfile</code> · <code>credits</code> · <code>favorites</code> · <code>siteVisits</code> · <code>changePassword</code> · <code>sessions</code> · <code>myListings</code> · <code>kyc.*</code> · <code>getAgents/getAgent</code> · <code>decorFavorites</code>
<code>subscriptions</code>	<code>plans</code> · <code>createOrder</code> · <code>verifyPayment</code> · <code>myCurrent</code> · <code>cancel</code> · <code>createPlan/updatePlan/deletePlan</code> · <code>adminList</code> · <code>gateway</code>
<code>admin</code>	<code>stats</code> · <code>users.*</code> · <code>properties.*</code> · <code>leads.*</code> · <code>teamMembers</code> · <code>creditUsage</code> · <code>buyerActivity</code> · <code>decorStores</code> · <code>KYC review</code>
<code>superAdmin</code>	<code>stats</code> · <code>systemHealth</code> · <code>security console</code> · <code>IP rules</code> · <code>permission matrix</code> · <code>billing</code> · <code>modelVersions</code> · <code>cmsPages</code> · <code>broadcastNotification</code>
<code>builders / projects</code>	directory browse + detail
<code>interiorDesigners</code>	<code>list</code> · <code>get</code> · <code>submit</code> · <code>favorites</code> · <code>recordView</code> · <code>unlockContact</code>
<code>decorStores</code>	<code>list</code> · <code>get</code> · <code>similar</code> · <code>submit</code> · <code>reportIssue</code> · <code>unlockContact</code> · <code>recordView</code>
<code>siteVisits</code>	<code>list</code> · <code>create</code> · <code>reschedule</code> · <code>cancel</code> · <code>complete</code> · <code>mapData</code>
<code>tickets</code>	<code>create</code> · <code>list</code> · <code>get</code> · <code>update</code> · <code>stats</code> · <code>report</code>
<code>reviews</code> · <code>searchAlerts</code> · <code>notifications</code> · <code>propertyViews</code>	CRUD + analytics for each
<code>reports</code>	snapshot (role-scoped) — live DB reporting across portals
<code>supervisor</code> · <code>campaigns</code> · <code>referrals</code> · <code>contact</code> · <code>media</code> · <code>siteContent</code>	team targets · marketing · referral rewards · contact form · R2 upload · CMS content

7.2 REST v1 (`/api/v1/*`)

A parallel Bearer-authenticated REST surface for external/integration use: leads (list/create/status/assign), listings, properties (list/get). 9 handlers across 6 files. Rate-limited at 100/min per IP via a shared `checkRateLimit()`.

8 · Auth & Security Model

A full defensive-posture pass (**GOL-268**, 07-04/07-05) closed all findings H1–H3, M1–M5, L1–L6. The resulting model:

Concern	How it works now
Session storage	Token in a signed, httpOnly cookie (not localStorage). Set server-side by <code>POST /api/auth/session</code> after login; re-verifies the token against the DB, never trusts client role. Clients send <code>credentials: "include"</code> .
Token at rest	<code>Session.token</code> stores <code>sha256(rawToken)</code> , never the raw value. Max 5 live sessions/user (oldest evicted).
Page-routing gate	<code>middleware.ts</code> (Node runtime) verifies an HMAC over <code>token role</code> before trusting the role for routing. Needs <code>SESSION_COOKIE_SECRET</code> .
CSP	<code>script-src</code> uses a per-request nonce (generated in middleware, threaded via <code>x-nonce</code>) instead of <code>unsafe-inline</code> . The 5 JSON-LD detail pages pass the nonce.
Rate limiting	tRPC middleware + all REST v1 handlers. Client IP resolved via <code>trustedClientIp()</code> using <code>TRUSTED_PROXY_COUNT</code> (default 1) — does not trust forgeable leftmost XFF hop.
Password	bcrypt cost 12. Staff vs consumer login separated.

Deploy gotcha: `SESSION_COOKIE_SECRET` must be set in the Vercel project env **before shipping** or all logins fail closed. The GOL-268 token-hashing change **invalidated all pre-existing sessions** (one-time; everyone re-logs in).

Deliberately deferred (product decisions, not bugs): real 2FA (fake toggle was removed — schema field kept unused), shortening the 30-day session TTL, rotating token on privilege change, and *enforcing* the permission-matrix levels in each route guard (matrix is currently authoring/simulation only).

9 · Core User Flows

9.1 Buyer: browse → unlock → lead

- 1 Buyer lands on `/` (featured carousel) or `/properties` (infinite scroll + filters).
- 2 Opens `/properties/[slug]` — SEO server-rendered detail with JSON-LD, image carousel, map pin, reviews.
- 3 Clicks **Unlock Owner Contact** → `properties.unlockContact` spends 1 credit → writes `CreditTransaction` → phone revealed. (0 credits → upsell to /pricing.)
- 4 Submits "Interested in this property?" form → `leads.create` → a `Lead(status=New)` enters the CRM.
- 5 Optionally **Schedule a Visit** → `siteVisits.create`; or saves via `users.addFavorite`.
- 6 Property views recorded on unmount with dwell time (`propertyViews.record` , anon-friendly).

9.2 Buyer: buy credits

- 1 `/pricing` loads plans from `subscriptions.plans` (DB is source of truth).
- 2 Checkout reads `subscriptions.gateway` (Razorpay or PayU) → `createOrder`.
- 3 Gateway widget/redirect → payment → `verifyPayment` (Razorpay signature) or `/api/payu/callback` (PayU hash).
- 4 On success: `Subscription` activated, credits topped up, `Payment` recorded, land on `/payment/success`.

9.3 Sales rep: work a lead

- 1 New lead appears in `/sales-portal` MyLeads (scoped to the rep via assignment).
- 2 Rep drags the lead across the CRM kanban (`@hello-pangea/dnd` , optimistic) → `leads.updateStatus`.
- 3 Click-to-call + `leads.addNote` log activity; schedules a `SiteVisit`.
- 4 On conversion → status Converted → drives `Commission`; rep-scoped reports via `reports.snapshot`.

9.4 Seller: list a property

- 1 Seller goes to `/list` (4-step form). Step 3 requires + validates a **RERA** number (`lib/rera.ts` , city → state regex).
- 2 Submit → `properties.create` (server re-validates RERA) → lands as `pending`.
- 3 Admin reviews in `/admin-portal` Listings → `properties.approve` → goes live.
- 4 Seller manages status/views/price in user-portal "My Listings" (`users.myListings`).

9.5 Admin / Super-admin: operate the platform

- 1 Admin approves listings, works the CRM pipeline, reviews KYC docs, manages subscriptions & team, watches credit-usage + buyer-activity analytics, runs campaigns.
- 2 Super-admin: command dashboard + system health, user/role management, security console (failed logins, IP rules, policy), permission matrix, plans manager, billing (MRR/ARR), AI model registry, CMS pages, audit trail, broadcast notifications (surface via the notification bell).

10 · What's Done SHIPPED

10.1 Platform & infra

- Turborepo monorepo, tRPC v11 wired end-to-end, Prisma 7 (engine-free) on live VPS PostgreSQL.
- Vercel deploy of `apps/web` (site + API); Prisma generated in `postinstall`.
- DB live: 39 tables pushed + seeded (7 users, 11 plans, 53 properties w/ galleries, leads, tickets, visits, subscriptions, 15 agents).
- Full **security hardening** (GOL-268): `httpOnly` signed cookie, hashed tokens, HMAC middleware gate, CSP nonce, rate-limited tRPC + REST v1.
- SEO: global metadata, `sitemap.xml` (ISR), `robots.txt`, per-listing `generateMetadata` + JSON-LD across 5 detail types; Search Console submitted.
- Mapbox: property-detail pin + supervisor geographic visit calendar; city-centroid fallback.
- Design system: branded Radix Select rolled across ~20 dropdowns, skeletons, load-more.

10.2 Public site & verticals

- Home, `/properties` (infinite + filters), `/properties/[slug]` (unlock, favorite, reviews, lead form, visit request).
- Auth: `/login`, `/register`, `/admin-login` → real DB.
- `/pricing` (DB plans + checkout), `/list` (4-step + RERA), `/profile`, `/contact`.
- Full verticals live from DB: Builders, Projects, Interiors (+ self-serve `/interiors/list`), Decor (+ `/decor/list`), Agents (+ `/agents/[slug]`).
- Legal/content pages; referral program; NRI guides; careers; unsubscribe; refund.

10.3 Portals wired to tRPC

- **User:** Overview, Saved, Credits (balance + top-up + active plan + usage), Profile, Visits (reschedule/cancel), Alerts (CRUD), My Listings, Recently Viewed, KYC upload, Support Tickets, Security panel (change password, sessions, sign-out).
- **Admin:** Stats, Listings (approve), CRM pipeline (drag-drop kanban), Leads, Team management, Subscriptions (cancel), KYC review, Property-view analytics, Credit-usage, Buyer-activity feed, Campaigns, Decor CRUD.
- **Sales:** MyLeads, click-to-call, rep-scoped Reports.
- **Supervisor:** TeamLeads, all-rep Reports, live geographic Visit Calendar.
- **Support:** Dashboard, Ticket Queue (filters/resolve/escalate/CSV), Escalations, My Assignments, TAT Report.
- **Super-admin:** Command dashboard + system health, User management, Security console, Notifications broadcast, Plans manager, Audit trail, Permission matrix (authoring/simulate), Billing, AI model registry, CMS.
- Notification bell (unread badge, poll, mark-read) in PortalShell + SiteHeader.

11 · What's Remaining ☐

11.1 Residual fake-data panels → tRPC HIGH PRIORITY

From `docs/static-data-inventory.md`. These still render from static fixtures:

Surface	Still fake	Action
/owners/[slug]	100% fake (static properties fixture)	Wire to DB
Home stats / reviews / press	Hardcoded incl. fake testimonials	SiteSetting CMS
Admin LeadsTab	Falls back to fake leads when DB empty (misleading)	Remove fallback
User portal panels	Recent views, featured, wallet ledger, disputes, viewed-CSV	DB
Fully fake tabs	sales Detail/Listings/Commission, supervisor Reassignment/ActivityMonitor, admin Commissions/Alerts	DB
Dead code	<code>lib/useDbData.ts</code> , dead exports in <code>static.ts</code>	Delete

11.2 Pricing single-source-of-truth MONEY RISK

Plan prices live in **4 places** (`subscriptions.ts`, `static.ts`, `pricingData.ts`, DB Plan table). Client shows static prices; checkout charges DB prices → **drift = user sees one price, pays another**. Consolidate: DB is truth, pricing page fetches `subscriptions.plans`.

11.3 Integrations needing live credentials

- **Payments** — `RAZORPAY_KEY_ID/SECRET` (and/or PayU keys) for real checkout.
- **Media** — `CLOUDFLARE_R2_*` for image/PDF uploads (media router ready).
- **WhatsApp** — `NEXT_PUBLIC_WHATSAPP_SALES/SUPPORT_NUMBER` (placeholders now).

11.4 Features not built

- Image/gallery upload UI on `/list`.
- Email / SMS OTP verification on register.
- Real **2FA** (secret/QR/verify) — fake toggle removed.
- EMI calculator (UI exists, calc static).
- Listing **Boost** (paid promotion — toast stub).
- Search-alert **match job** (no backend matcher → no match counts).
- **Enforce** permission-matrix levels in route guards (currently authoring-only).
- Business numbers → CMS/DB (referral rewards, monthly target, TAT hours).
- Move marketing/footer content → SiteSetting CMS (pattern exists).

11.5 Suggested order of attack

1. Pricing dedup (money correctness).
2. Portal fixtures → tRPC (biggest fake-data surface).
3. Home/footer content → SiteSetting CMS.
4. Env placeholders (one-line fixes; need real values).
5. Dead-code cleanup.

12 · Known Issues & Tech Debt

Item	Detail
VPS-as-prod hardening	Open port 5433 to Vercel (dynamic IPs → effectively public), add <code>?sslmode=require</code> (traffic currently unencrypted), add PgBouncer before real traffic (serverless connection limits).
Serverless pooling	<code>pg.Pool</code> opens per function instance — cap pool <code>max</code> + use pooled URL before real traffic.
apps/api type-check fails	Pre-existing (reproduces before this session). Module-resolution mismatch vs extensionless imports in packages. Undeployed, so not blocking — flagged as separate task.
SiteHeader blur / dropdown	Same containing-block bug as GOL-270 (fixed in PortalShell) still latent on the sticky SiteHeader where the blur is load-bearing. Follow-up.
Social links	Footer socials are live <code>href="#"</code> placeholders — fix or hide.
No migrations	Schema via <code>db push</code> only — no migration history/rollback. Fine for prototype; revisit for prod.

Source of truth for debt: `progress.md` (chronological build log) and `docs/static-data-inventory.md` (fake-data audit). Both are kept current — update them when you ship.

13 · Local Setup & Dev Workflow

13.1 Prerequisites

- Node ≥ v20 · pnpm ≥ v9 · (Docker for local Postgres/Redis/Typesense — optional, since local can hit the shared VPS DB)

13.2 First run

Step	Command
Install	<code>pnpm install</code>
Env	<code>cp .env.example .env --set DATABASE_URL (password @ → %40), SESSION_COOKIE_SECRET , NEXT_PUBLIC_MAPBOX_TOKEN</code>
Prisma client	<code>pnpm run db:generate</code>
Dev servers	<code>pnpm run dev --> web :3000, api :3001</code>
DB browser	<code>pnpm run db:studio</code>

Do not run `migrate dev` — shared DB. Additive schema changes: edit `schema.prisma` → `prisma db push` → `db:generate`.

13.3 Adding a tRPC procedure

1. Put reusable input schema (Zod) in `packages/shared`, or inline in the router.
2. Add the query/mutation in `packages/trpc/src/routers/<router>.ts` with the right procedure guard.
3. Call it in the frontend: `trpc.<router>.<procedure>.useQuery(...)` — fully typed.

13.4 Pre-commit checklist

- `pnpm --filter @nx-tsft/web build` — zero TS/route errors
- `pnpm type-check` + `pnpm lint` (note: apps/api type-check is a known failure)
- Browser-verify the flow you touched (see progress.md E2E checklist)

Demo logins (all password `demo1234`): `rohan@example.com` (buyer), `priya@nx-tsft.com` (sales), `supervisor@nx-tsft.com`, `support@nx-tsft.com`, `admin@nx-tsft.com`, `sa@nx-tsft.com` (super-admin).

14 · Deployment

- **Target:** Vercel, project **Root Directory** = `apps/web` . Only the web app deploys (site + API).
- **Prisma:** generated on Vercel via `packages/db` postinstall. Prisma 7 = no native engine to bundle.
- **Required Vercel env:** `DATABASE_URL` (pooled, %40-encoded), `SESSION_COOKIE_SECRET` (**or logins fail closed**), `NEXT_PUBLIC_MAPBOX_TOKEN` , `NEXT_PUBLIC_SITE_URL` , payment + R2 keys when live.
- **Canonical host:** `https://www.nxtsft.com` (apex 308-redirects to www). Sitemap must be submitted as the **www** URL.
- Full guide: `docs/DEPLOYMENT.md` .

15 · First-Week Onboarding Path

- 1 **Read this doc** → `ARCHITECTURE.md` → `progress.md` → `docs/static-data-inventory.md`. These four give you the full picture.
- 2 **Run it:** `pnpm install` → set `.env` → `pnpm run dev`. Log in as each demo role and click through all six portals.
- 3 **Trace one flow end-to-end:** browse → unlock contact → lead created. Follow it from `/properties/[slug]` UI → `properties.unlockContact` router → `CreditTransaction / Lead` in Prisma Studio.
- 4 **Explore the graph:** this repo has a code-review knowledge graph (MCP tools) — use it before grepping to see callers/dependents/tests.
- 5 **Pick a starter task:** a §11.1 fake-panel → tRPC swap is the ideal first PR — self-contained, visible, and the pattern (a wired panel) already exists to copy.
- 6 **Before you push:** build + type-check + browser-verify the touched flow; then update `progress.md` and the inventory doc.

The one rule from `CLAUDE.md`: Ask, don't assume. Implement the simplest thing that solves the problem, don't touch unrelated code (but do surface smells), and flag uncertainty explicitly.